

Application Note: Current Exact Code Path for Tag Read/Filter/Write

Date: 2026-05-21

This document is rewritten to match the exact current implementation in:

- app/src/main/java/com/zebra/rfid/demo/sdksample/RFIDHandler.kt
- app/src/main/java/com/zebra/rfid/demo/sdksample/MainActivity.kt

Actual Runtime Behavior (Current)

1. The write dialog button label says "Write with TID Filter".
2. Current `writeWithTidPrefilter(...)` directly calls `handler.write(...)`.
3. The TID-prefilter read/apply/write block in `writeWithTidPrefilter(...)` is currently commented out.
4. `RFIDHandler.write(...)` and `RFIDHandler.read(...)` both support empty tag ID by converting to `effectiveTagID = null`.
5. `RFIDHandler.read(...)` uses `readWait(..., true)` so inventory filter/singularization can be respected.

MainActivity Exact Methods

buildTidFilterHex

```
```kotlin
private fun buildTidFilterHex(rawTid: String): String {
 val hexOnly = rawTid
 .replace(" ", "")
 .uppercase(Locale.US)
 .filter { it in "0123456789ABCDEF" }

 // 32 hex chars = 128 bits
 return if (hexOnly.length > 32) hexOnly.substring(0, 32) else hexOnly
}...
```

#### ### writeWithTidPrefilter (exact current code)

```
```kotlin
private fun writeWithTidPrefilter(
    targetTagId: String,
    data: String,
    selectedBank: MEMORY_BANK,
    offset: Int,
    tidWordCount: Int = 8
) {
    val handler = rfidHandler
    if (handler == null) {
        sendToast("RFID handler unavailable")
        return
    }

    if (targetTagId.isBlank()) {
        sendToast("Tag ID is empty")
        return
    }

    if (data.isBlank() || data.length % 4 != 0) {
        sendToast("Write data must be hex and length multiple of 4")
        return
    }

    handler.write(targetTagId, data, selectedBank, offset)
```

```

//      handler.read(targetTagId, MEMORY_BANK.MEMORY_BANK_TID, 0, tidWordCount
) { tid ->
//          if (tid.isNullOrBlank()) {
//              sendToast("Read TID failed")
//              return@read
//          }
//
//          val tidFilter = buildTidFilterHex(tid)
//          if (tidFilter.isBlank()) {
//              sendToast("Invalid TID for prefilter")
//              return@read
//          }
//
//          sendToast("TID prefilter applied, writing tag...")
//          handler.setSingulationForFilter(true)
//          handler.applyTagFilter(tidFilter)
//          handler.write(targetTagId, data, selectedBank, offset)
//      }
}
}

```

RFIDHandler Exact Methods

applyTagFilter

```

```kotlin
public fun applyTagFilter(tag: String) {
 val currentReader = reader
 if (currentReader == null) {
 Log.e(TAG, "RFIDReader is null")
 return
 }
 //E280119120007676B2EF034A000000009000

 // Add state aware pre-filter
 val filters = PreFilters()
 // In Kotlin, inner classes are instantiated by calling the constructor on t
he outer instance
 val filter = filters.PreFilter()

 filter.setAntennaID(1.toShort()) // Explicitly cast to Short in Kotlin
 filter.setTagPattern(tag)
 if(tag.length*4 > 128)
 filter.setTagPatternBitCount(128)
 else
 filter.setTagPatternBitCount(tag.length * 4)

 filter.setBitOffset(0) // skip PC bits (always it should be in bit length)
 filter.setMemoryBank(MEMORY_BANK.MEMORY_BANK_TID)
 filter.setFilterAction(FILTER_ACTION.FILTER_ACTION_STATE_AWARE)

 // inventoried flag of session S1 of matching tags to B
 filter.StateAwareAction.setTarget(TARGET.TARGET_INVENTORIED_STATE_S0)

 // not to select tags that match the criteria
 filter.StateAwareAction.setStateAwareAction(STATE_AWARE_ACTION.STATE_AWARE_A
CTION_INV_B_NOT_INV_A)

 try {
 // Clear existing pre-filters before adding new ones
 currentReader.Actions.PreFilters.deleteAll()
 }
}

```





